Pet Adoption System

Group 14



組員:
1131630 孫文芯、1131631 郭璟恩、1131651 許熙言、1131653 江佳蓁

專案目標 projet goal

專案目標:

本專案目的為運用spring boot框架，開發一套基於微服務架構的寵物領養管理系統

核心目標:

1．管理從「領養人申請領養」到「寵物領養手續完成」之流程

2．管理領養人資料、待領養動物資料以及領養者資訊驗證系統

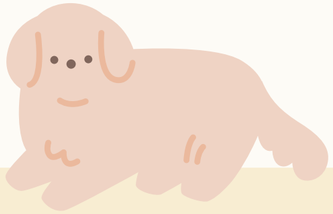
系統架構



Service A - Pet Service (核心寵物服務)

目的:

- 為資料中心供其他服務查詢寵物即時狀態



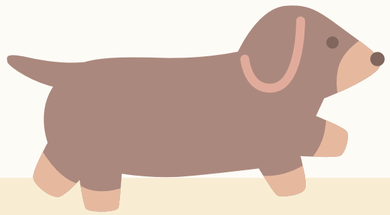
Service B - Adoption Service (領養管理服務)

目的:

- 負責處理領養申請的生命週期，從提交申請、審核到最終完成
- 確保領養流程的公平性與正確性，並協調使用者與寵物之間的關係

系統架構





Service C - User Service (使用者服務)

目的:

- 管理領養者 (User/Adopter) 的基本資料與信用評價
- 提供資格審核機制，防止黑名單用戶或惡意重複申請

Service D - Shelter Service (收容所服務)

說明:

- 管理各個收容所的機構資料。

系統架構



service E - Authentication Service (驗證服務)

目的:

- 處理會員註冊與登入驗證
- 讓成功登入的會員能進行申請或修改個資

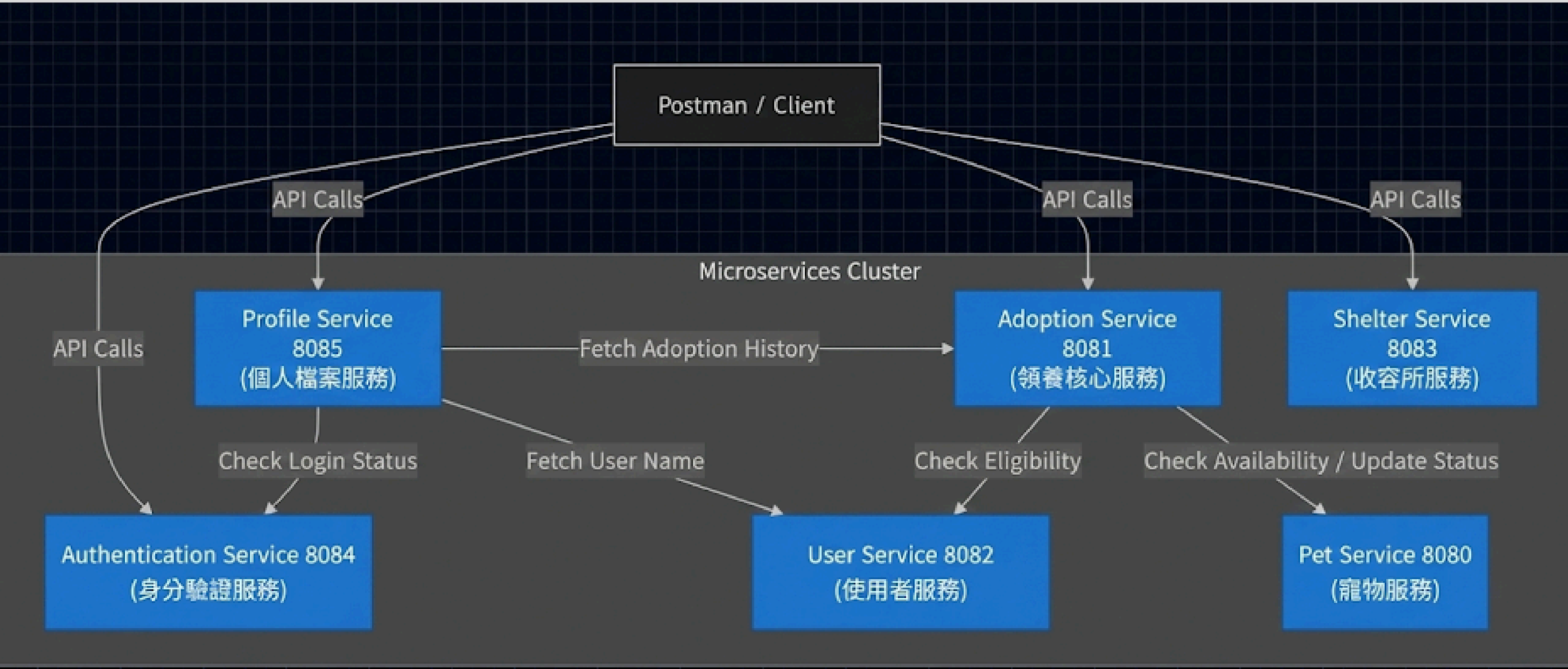


service F - Profile Service (個人資料/偏好服務)

目的:

- 整合會員的個人資料、領養偏好與歷史紀錄
- 提供會員單一視圖來管理自己的領養生涯與偏好設定

流程圖


Postman / Client
API Calls
API Calls
API Calls
API Calls
Microservices Cluster
Profile Service
8085
(個人檔案服務)
Fetch Adoption History
Adoption Service
8081
(領養核心服務)
Shelter Service
8083
(收容所服務)
Check Login Status
Fetch User Name
Check Eligibility
Check Availability / Update Status
Authentication Service 8084
(身分驗證服務)
User Service 8082
(使用者服務)
Pet Service 8080
(寵物服務)

Service A - Pet Service (核心寵物服務)



核心功能:

- 寵物資料 RESTful API (CRUD)
- 狀態流轉控制與防呆邏輯 (防止刪除申請中寵物)

架構互動:

- 作為被呼叫端 (Provider) 供 Adoption Service 串接
- 提供可用性檢查 (/isAvailable) 與狀態鎖定

Java 技能:

- Spring Boot Framework (DI, REST)
- Global Exception Handling (統一錯誤處理)
- Java I/O Serialization (檔案存取)
- Java 8 Stream API & Optional (資料流處理)

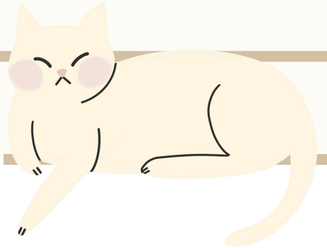
Service B - Adoption Service (領養管理服務)

核心功能:

- 全生命週期管理：處理申請、審核至完成的完整流程
- 嚴格資格審查：建立申請前同步驗證「使用者資格」與「寵物狀態」，防止無效申請
- 數據統計：即時計算領養成功率

架構互動:

- WebClient 呼叫端：作為協調者串接微服務
- User Service 整合：確認使用者是否符合領養資格
- Pet Service 同步：遠端控制寵物狀態 (鎖定 HOLD / 完成 ADOPTED)



Java 技能:

- Spring WebFlux：使用 WebClient 處理 HTTP 請求與回應
- Advanced REST Client：動態 URL 建構 (UriBuilder) 與同步阻斷 (block()) 處理
- Java 8 Stream API：高效處理集合資料統計
- Custom Exception Handling：將跨服務錯誤轉為自定義異常

Service C - User Service (使用者服務)



核心功能:

- 使用者資料管理 (CRUD)：提供註冊、查詢與資料更新 API
- 資格驗證機制：內建防護邏輯，檢查黑名單 (Blacklist) 狀態與申請額度 (上限 3 筆)

架構互動:

- 服務提供者：作為被呼叫端，不依賴其他微服務
- 支援領養決策：開放 /eligible 接口供 Adoption Service 同步呼叫，決定是否放行申請

Java 技能:

- OOP 封裝邏輯：將資格判斷 (isEligible) 封裝於 Model 層，落實物件導向設計
- 標準化異常回應：使用 @RestControllerAdvice 回傳包含 Timestamp 與 HTTP Status 的結構化 JSON
- Java Stream API：使用 filter 與 findFirst 進行高效資料檢索

Service D - Shelter Service (收容所服務)



核心功能:

- 完整 CRUD 管理：提供收容所資料的新增、查詢、修改 (PUT) 與刪除功能
- 資料初始化 (Bootstrapping)：服務啟動時自動載入預設資料 (如：中壢/內壢收容所)
- 錯誤狀態對應：查詢不存在 ID 時，自動對應 HTTP 404 Not Found

架構互動:

- 基礎資訊提供者：作為被維護機構實體資訊
- 資料關聯中心：提供收容機構基礎資訊，供未來寵物檔案擴充關聯使用

Java 技能:

- Spring Web MVC：熟練運用 @GetMapping, @PostMapping, @PutMapping, @DeleteMapping 定義語意化路由
- Java Stream API：使用 stream().filter().findFirst() 優化集合搜尋邏輯
- Annotation-Driven Exception：使用 @ResponseStatus 簡化異常類別與 HTTP 狀態碼的綁定

Service E - Authentication Service (驗證服務)



核心功能:

- 身分驗證：處理使用者註冊 (/register) 與登入 (/login) 邏輯，包含密碼比對
- Session 狀態管理：維護使用者線上狀態，支援登入狀態維護與狀態查詢
- 異常控制：針對帳號不存在或密碼錯誤，拋出 LoginFailedException 並回傳 400 錯誤

架構互動:

- 安全守門員 (Gatekeeper)：作為系統的單一認證入口點
- 狀態提供者 (State Provider)：開放 /isLoggedIn 接口，供其他微服務 (如 Profile Service) 在執行敏感操作前確認使用者的登入狀態

Java 技能:

- In-Memory Caching：使用 static Map 與 List 實作輕量級的 Session 暫存機制
- Custom Exception Handling：自定義 LoginFailedException 結合 @RestControllerAdvice 統一處理登入失敗邏輯
- Java Optional API：使用 orElseThrow 優化 Null Safety 的查詢邏輯

Service F - Profile Service (個人資料/偏好服務)



核心功能:

- 進階個資管理：維護緊急聯絡人、地址等詳細資訊，支援「若無則新增」邏輯
- 領養偏好設定 ：允許使用者設定偏好的寵物類型、年齡與體型，作為未來推薦系統的基礎
- 權限與歷史整合：彙整來自 Adoption Service 的歷史紀錄

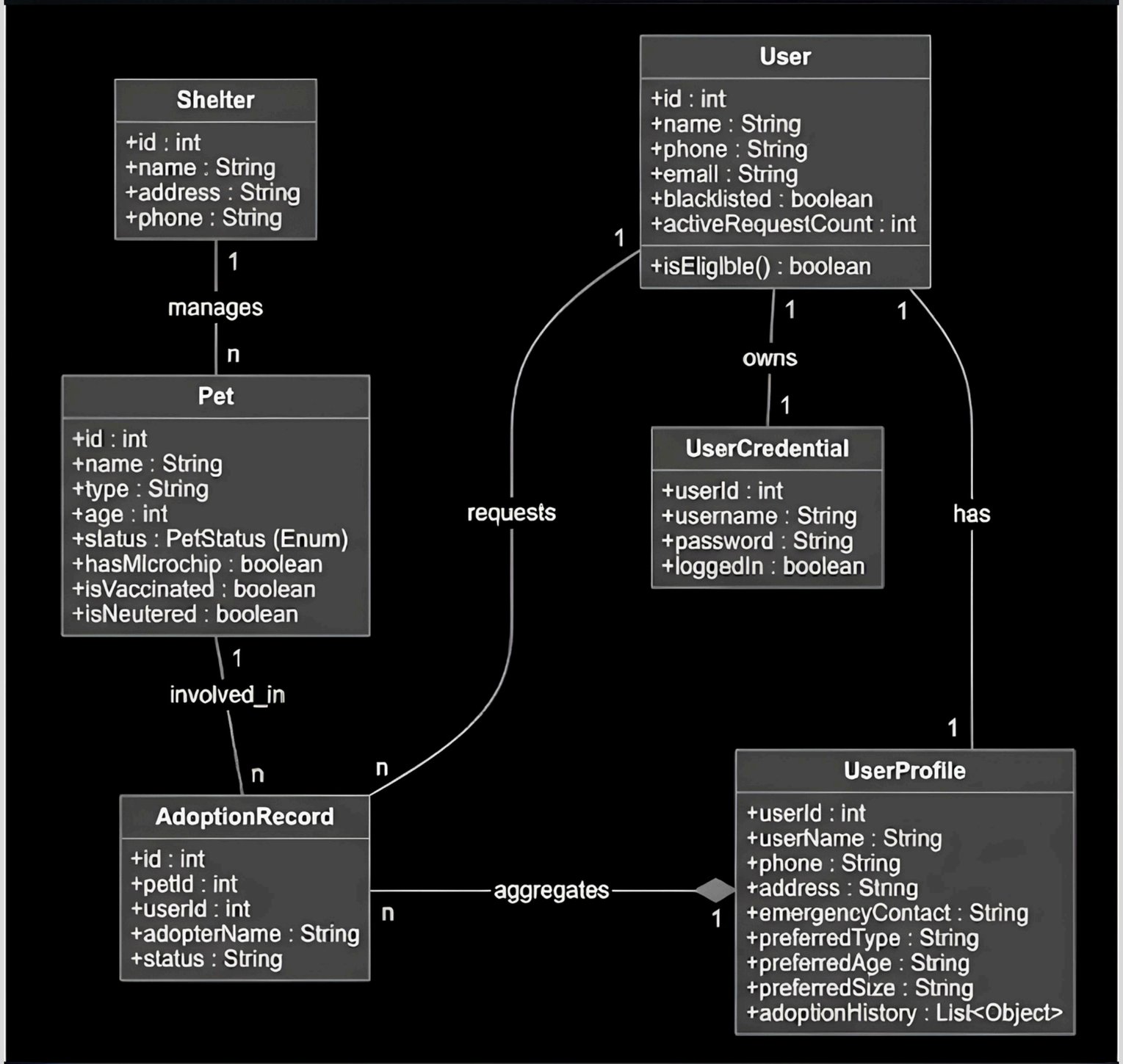
架構互動:

- 服務呼叫端 (Consumer)：
- 呼叫 Auth Service：確認操作者目前的登入狀態
- 呼叫 Adoption Service：取得跨服務的領養歷史資料

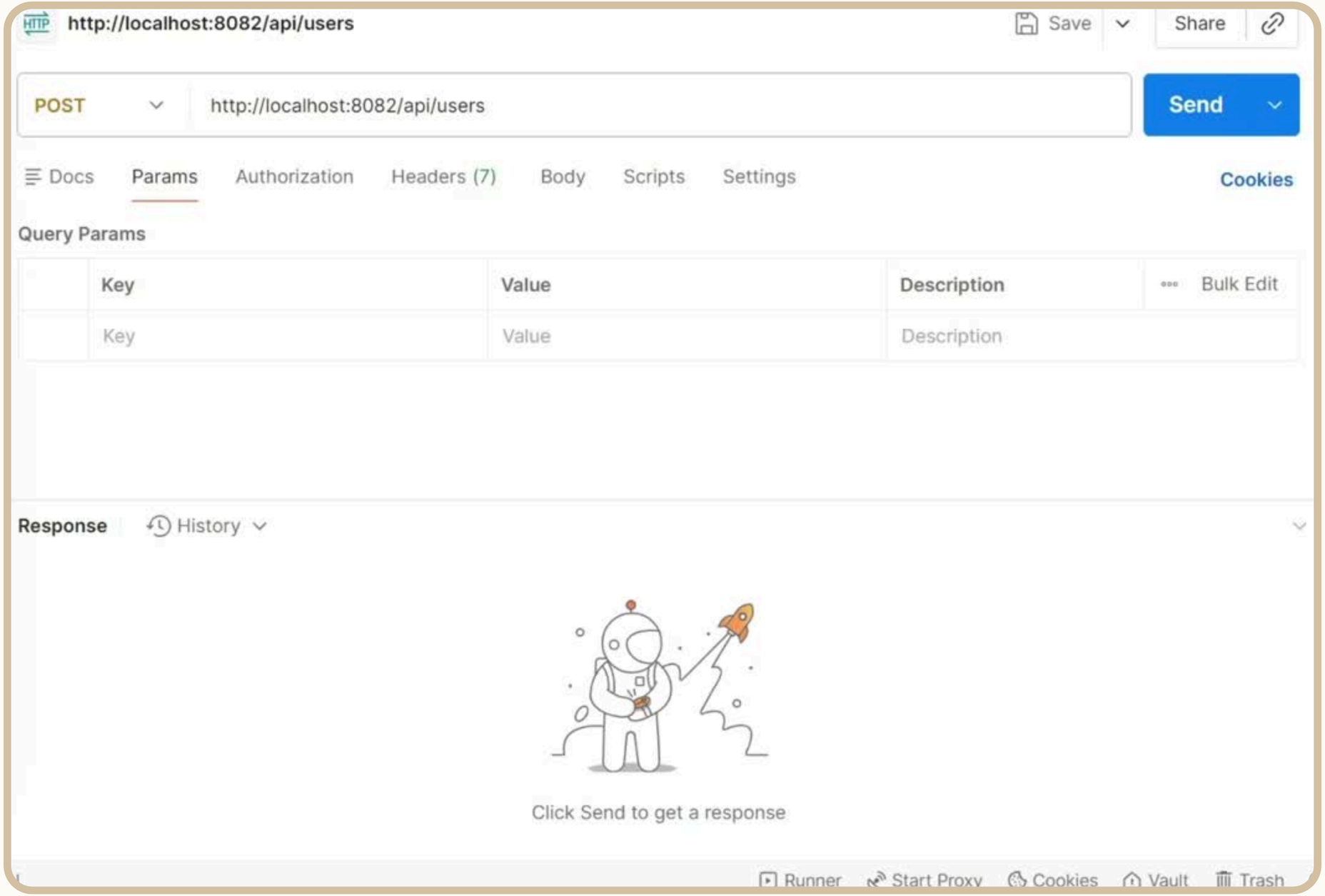
Java技能:

- RESTful PATCH：精確使用 @PatchMapping 實作資源的「部分更新」語義
- Dynamic JSON Handling：使用 Map<String, String> 彈性解析 Request Body，解決參數傳遞問題
- Security Exception：實作 PermissionDeniedException 並對應至 HTTP 403 Forbidden 狀態碼
- Optional Functional Style：使用 orElseGet() 優雅處理資料初始化邏輯

系統物件設計圖
Object Design



Spring boots 測試影片



前端畫面展示

← 回前台首頁

管理員後台

功能選單

請選擇要管理的項目：

寵物管理

新增、刪除、查看所有寵物狀態（AVAILABLE / HOLD / ADOPTED）。

前往寵物管理

領養申請審核

查看所有領養申請，核准 / 完成流程，控制寵物最終狀態。

前往申請審核

會員管理

查看會員基本資料、黑名單狀態與申請次數。

遇到的困難

關於說明文件:	從零開始思考和決定每個service內要有甚麼功能以及要運用到那些物件和功能時有點不知該從何下手，透過不斷地交換意見和討論才決定了每一階段的設計文件內容
關於java程式碼:	本來以為所有service都要寫在同一個檔案夾內，後來發現這樣做會有問題於是後來決定分開一個個service寫
關於前後端:	單看設計文件跟後端的測試結果，對前端的畫面設計沒有頭緒，一開始做出來的與後端測試畫面差很多
關於spring boots:	一開始不確定spring boots要如何使用以及它的方便性在哪，因此需花費時間學習
關於 postman :	postman 在測試時常常會發生測試結果與預期不符的狀況，因此要常常修改程式或是重複測試確認是測試流程還是程式碼的問題

使用AI協助的地方

關於說明文件:	利用AI協助將每個service可能會用到的程式與物件分門別類
關於java程式碼:	使用AI確認每個java程式與service是否匹配或有功能上的缺漏，以及對程式進行修改與調整
關於前端設計:	利用AI協助JavaScript 儲存資料、互動的程式撰寫
關於spring boots:	使用gemini學習如何使用spring boots將我們寫的功能放進spring boots為我們準備的架構模板裡
關於 postman :	在API測試過程中常遇到查不到資料、執行錯誤或輸出不如預期的情況，因此我們透過詢問Gemini在不清除舊檔案的前提下，找到正確的數據來產出我們想要的測試報告

小組分工

成員	分工
孫文芯	postman測試與程式碼調整
郭璟恩	說明文件、簡報製作與解說
許熙言	html前端製作
江佳蓁	java程式碼寫作與調整